

下单域接口规范

1. 概览

1.1 路径列表

方法	路径	说明	对齐
POST	/api/v1/portfolio/orders	创建注单 (Single 或 Parlay)	Kalshi POST /portfolio/orders
POST	/api/v1/portfolio/orders_bulk	批量创建 (≤ 15)	Kalshi /orders_bulk / Polymarket postOrders
POST	/api/v1/portfolio/orders/cancel	撤单 (三模式)	Kalshi DELETE /portfolio/orders/{id}
GET	/api/v1/portfolio/orders	订单列表	Kalshi GET /portfolio/orders
GET	/api/v1/portfolio/orders/get	单查	Kalshi GET /portfolio/orders/{id}

老的 /api/v1/bet/* 路径同步保留，避免破坏存量联调，字段语义完全一致。

1.2 认证

- Phase 1: account_id 走请求体，由网关层鉴权后注入；服务自身不做 JWT 校验
- Phase 2: 迁移到 Kalshi 风格 KALSHI-ACCESS-KEY / KALSHI-ACCESS-SIGNATURE / KALSHI-ACCESS-TIMESTAMP 三段式 RSA-PSS 签名

1.3 通用响应封装

所有接口的成功 / 业务失败均返回 HTTP 200 + 业务码：

```
1 {
2   "code": 0,
3   "msg": "ok",
4   "data": { /* 具体响应体 */ }
5 }
6
```

- code = 0 → 成功
- code 在 40000-49999 区间 → 业务失败（详见 §7 错误码表）
- code 在 50000-59999 区间 → 系统失败，HTTP 500

2. 数据模型

2.1 Order（订单）

字段	类型	说明
order_id	int64	服务端注单 ID (Snowflake)
client_order_id	string	客户端幂等键；服务端 (account_id, client_order_id) 唯一
order_group_id	string	同组订单标识。Single 多笔自动归到 grp_{client_order_id}；Parlay 整张共用
account_id	int64	账户 ID
subaccount	int	子账户号，0 = 主账户
order_type	enum	single / parlay (差异项 ④)
action	enum	buy (Phase 1 唯一支持，差异项 ②)
time_in_force	enum	fill_or_kill (Phase 1 唯一支持，差异项 ③)

coin	string	计价币种，当前 USDT
notional_dollars	decimal	押注金额（权威字段）
decimal_odds	decimal	欧式赔率：Single = leg odds，Parlay = 各 leg 连乘（权威字段）
yes_price_dollars	decimal	派生： $= 1 / \text{decimal_odds}$ (Kalshi 兼容)
count_fp	decimal	派生： $= \text{notional_dollars} \times \text{decimal_odds}$ (Kalshi 兼容；等价合约份数)
max_payout_dollars	decimal	$= \text{notional} \times \text{decimal_odds}$ ，最大派奖（含本金）
payout_dollars	decimal	实际派奖；结算前为 0
status	enum	resting / cancelled / executed (Kalshi 风格对外状态)
outcome	enum	won / lost / refunded / cancelled / rejected（差异项⑤）
biz_state	enum	内部业务态： Pending / Won / Lost / Refunded / Cancelled
chain_state	enum	内部链上态： Pending / Submitt

		ed / Confirmed / Failed
bet_tx_hash	string	下注上链 hash
cancel_tx_hash	string	撤单上链 hash
payout_tx_hash	string	派奖上链 hash
reject_code	int	失败时的业务错误码
reject_reason	string	失败时的文案
legs	OrderLeg[]	注单腿明细，见 §2.2
created_at	RFC3339	创建时间
updated_at	RFC3339	更新时间
settled_at	RFC3339	结算时间；未结算为零值

2.1.1 status ↔ (biz_state, chain_state) 映射

status	outcome	触发条件
resting	""	biz_state=Pending 且 chain_state ∈ {Pending, Submitted, Confirmed}
canceled	cancelled	biz_state=Cancelled
canceled	rejected	biz_state=Pending 且 chain_state=Failed (链上提交失败)
executed	won	biz_state=Won
executed	lost	biz_state=Lost

executed	refunded	biz_state=Refunded
----------	----------	--------------------

客户端只需读 status + outcome，不需要解析内部态。biz_state / chain_state 仅用于排查问题。

2.2 OrderLeg (注单腿)

字段	类型	说明
leg_id	int64	腿 ID
ticker	string	{match_id}. {market_code}. {option_key}，全局唯一
selection_id	int64	内部选项 ID
match_id	int64	比赛 ID
market_id	int64	市场 ID
option_key	string	选项编码，如 home / over / score:2-1
decimal_odds	decimal	锁定时的欧式赔率
yes_price_dollars	decimal	= 1 / decimal_odds
outcome	enum	Pending / Won / Lost / Push / Void / HalfWin / HalfLoss
seq_no	int	在该订单内的序号（从 1 开始）

2.3 ticker 格式说明

```
1 {match_id}.{market_code}.{option_key}
2
```

例：

ticker	含义
100123.result_1x2_ft.home	比赛 100123 的「胜平负 - 主胜」
100123.over_under_ft.over	比赛 100123 的「合计 - 大球」
100123.asian_handicap_ft.home	比赛 100123 的「亚洲让分盘 - 主队」
100123.correct_score_ft.score:2-1	比赛 100123 的「正确进球 - 2:1」

与 Kalshi 单段全局唯一字符串不同，我们走三段，因为（`market_code`, `option_key`）在不同比赛中复用，必须带 `match_id` 才能唯一定位。

`market_code` 在 Phase 1 的取值：
`result_1x2_ft` / `kickoff` / `asian_handicap_ft` / `euro_handicap_ft` / `goals_count_ft` / `over_under_ft` / `correct_score_ft`。

3. 创建注单

3.1 接口

```
1 POST /api/v1/portfolio/orders
2 Content-Type: application/json
3
```

3.2 请求体

字段	类型	必填	说明
<code>client_order_id</code>	string	是	幂等键；同账户重复请求直接返回旧订单
<code>account_id</code>	int64	是	账户 ID
<code>coin</code>	string	是	计价币种，当前固定 <code>USDT</code>

order_type	enum	是	single 或 parlay
legs	OrderLegInput[]	是	至少 1 条； parlay 时至少 2 条
notional_dollars	decimal	条件	parlay 必填； single 与 per_leg_notional_dollars 二选一
per_leg_notional_dollars	decimal[]	条件	single 多笔时按 leg 一一对应；缺省走等额拆分
subaccount	int	否	默认 0
time_in_force	enum	否	默认 fill_or_kill，其他值会被拒绝
action	enum	否	默认 buy，其他值会被拒绝
order_group_id	string	否	不填时服务端自动生成 grp_{client_order_id}
expiration_ts	int64	否	报价超时（unix 秒）；0 表示无超时
accepted_at	RFC3339	否	客户端"接受最新报价"的时间，与 expiration_ts 二选一即可

OrderLegInput

字段	类型	必填	说明
ticker	string	条件	与 selection_id 二选一；优先使用
selection_id	int64	条件	内部 ID 兼容字段
decimal_odds	decimal	条件	锁定的欧式赔率；与 yes_price_dollars 二选一
yes_price_dollars	decimal	条件	Kalshi 风格价格；服务端反推 $\text{decimal_odds} = 1 / \text{yes_price_dollars}$

3.3 响应体

data 字段是 OrderView[] :

- order_type=single 多笔提交时：返回 N 条 OrderView
- order_type=single 单笔 / parlay：返回 1 条 OrderView

3.4 校验顺序（服务端实现的全部校验）

1. 基础参数 (leg 数、client_order_id 非空、time_in_force=fill_or_kill、action=buy)
2. 报价 TTL (expiration_ts 或 accepted_at 任一过期均拒绝)
3. 幂等查找：(account_id, client_order_id) 已存在则直接返回旧记录，不进入后续流程
4. 解析每条 leg (ticker → selection_id)
5. 同盘口替换：同一注单内同 market_id 不能出现两次
6. 比赛状态：必须 Scheduled (开赛后封盘)
7. 市场状态：必须 Open
8. 赔率 CAS：decimal_odds 必须等于服务端当前 current_odds，否则 41003

- 9. 串关同场互斥 (仅 `parlay`) : 见 §3.5
- 10. 押注金额范围 : `min_stake ≤ notional ≤ max_stake`
- 11. 余额预校验 (链下镜像, 不授权)
- 12. 创建草稿订单 → 提交链上

3.5 串关同场互斥矩阵

仅 `order_type=parlay` 触发 ; 多笔单注 (多个 `single`) 不受影响。

- 同 `match_id` 下的多条 `leg` 必须满足互斥矩阵
- `kickoff` (开球权) 与任意盘口可组合
- 其余强相关盘口对禁止同场组合 :

盘口 A	盘口 B	是否允许同场串关
<code>result_1x2_ft</code>	<code>asian_handicap_ft</code> / <code>euro_handicap_ft</code> / <code>correct_score_ft</code>	否
<code>asian_handicap_ft</code>	<code>euro_handicap_ft</code> / <code>correct_score_ft</code>	否
<code>euro_handicap_ft</code>	<code>correct_score_ft</code>	否
<code>goals_count_ft</code>	<code>over_under_ft</code> / <code>correct_score_ft</code>	否
<code>over_under_ft</code>	<code>correct_score_ft</code>	否
其他对	—	是

冲突时返回 `41006`。

3.6 示例

3.6.1 单笔单注 (Single)

请求 :

```
1 POST /api/v1/portfolio/orders
2 {
3   "client_order_id": "cli_20260509_001",
4   "account_id": 1001,
5   "coin": "USDT",
```

```

6  "order_type": "single",
7  "notional_dollars": "100",
8  "legs": [
9    {
10     "ticker": "100123.result_1x2_ft.home",
11     "decimal_odds": "1.85"
12   }
13 ]
14 }
15

```

响应：

```

1  {
2    "code": 0,
3    "msg": "ok",
4    "data": [
5      {
6        "order_id": 7283901020001,
7        "client_order_id": "cli_20260509_001",
8        "order_group_id": "grp_cli_20260509_001",
9        "account_id": 1001,
10       "subaccount": 0,
11       "order_type": "single",
12       "action": "buy",
13       "time_in_force": "fill_or_kill",
14       "coin": "USDT",
15       "notional_dollars": "100",
16       "decimal_odds": "1.85",
17       "yes_price_dollars": "0.540541",
18       "count_fp": "185.00",
19       "max_payout_dollars": "185",
20       "payout_dollars": "0",
21       "status": "resting",
22       "biz_state": "Pending",
23       "chain_state": "Submitted",
24       "bet_tx_hash": "STUB-bet-7283901020002",
25       "legs": [
26         {
27           "leg_id": 7283901020003,
28           "ticker": "100123.result_1x2_ft.home",
29           "selection_id": 5001,
30           "match_id": 100123,
31           "market_id": 901,
32           "option_key": "home",
33           "decimal_odds": "1.85",
34           "yes_price_dollars": "0.540541",
35           "outcome": "Pending",
36           "seq_no": 1
37         }
38       ],
39       "created_at": "2026-05-09T15:08:21+08:00",
40       "updated_at": "2026-05-09T15:08:21+08:00"
41     }
42   ]
43 }
44

```

3.6.2 多笔单注 (Single × N, 等额拆分)

请求：

```

1  {
2    "client_order_id": "cli_20260509_002",
3    "account_id": 1001,

```

```

4   "coin": "USDT",
5   "order_type": "single",
6   "notional_dollars": "300",
7   "legs": [
8     { "ticker": "100123.result_1x2_ft.home",    "decimal_odds": "1.85" },
9     { "ticker": "100124.over_under_ft.over",    "decimal_odds": "2.10" },
10    { "ticker": "100125.result_1x2_ft.draw",     "decimal_odds": "3.20" }
11  ]
12 }
13

```

- 服务端拆分为 3 张独立 ticket，每张 notional=100
- 子单 client_order_id = cli_20260509_002#1 / #2 / #3，便于客户端追溯
- 整组共享 order_group_id = grp_cli_20260509_002
- 任一腿失败不影响其他腿

3.6.3 多笔单注 (Single × N，按 leg 指定金额)

```

1  {
2    "client_order_id": "cli_20260509_003",
3    "account_id": 1001,
4    "coin": "USDT",
5    "order_type": "single",
6    "per_leg_notional_dollars": ["50", "200", "30"],
7    "legs": [ /* 3 条 */ ]
8  }
9

```

3.6.4 串关 (Parlay)

请求：

```

1  {
2    "client_order_id": "cli_20260509_004",
3    "account_id": 1001,
4    "coin": "USDT",
5    "order_type": "parlay",
6    "notional_dollars": "50",
7    "legs": [
8      { "ticker": "100123.result_1x2_ft.home", "decimal_odds": "1.85" },
9      { "ticker": "100124.over_under_ft.over", "decimal_odds": "2.10" },
10     { "ticker": "100125.kickoff.home",        "decimal_odds": "1.95" }
11   ]
12 }
13

```

响应 (节选)：

```

1  {
2    "order_type": "parlay",
3    "notional_dollars": "50",
4    "decimal_odds": "7.5757500",
5    "max_payout_dollars": "378.79",
6    "legs": [ /* 3 条 */ ]
7  }
8

```

- decimal_odds = 1.85 × 2.10 × 1.95 = 7.575750
- 任一腿 Lost → 整单 Lost

- 任一腿 Void → 该腿按赔率 1.0 计入，重新连乘
- 全部 Void → 退本

3.6.5 用 Kalshi 风格 yes_price 下单

```

1 {
2   "client_order_id": "cli_20260509_005",
3   "account_id": 1001,
4   "coin": "USDT",
5   "order_type": "single",
6   "notional_dollars": "100",
7   "legs": [
8     {
9       "ticker": "100123.result_1x2_ft.home",
10      "yes_price_dollars": "0.5405"
11    }
12  ]
13 }
14

```

服务端反推 $\text{decimal_odds} = 1 / 0.5405 \approx 1.8501$ ，再走赔率 CAS。

由于浮点精度，强烈建议生产环境直接传 `decimal_odds`；
`yes_price_dollars` 仅作为 Kalshi SDK 兼容入口。

4. 批量创建

4.1 接口

```

1 POST /api/v1/portfolio/orders_bulk
2 Content-Type: application/json
3

```

4.2 请求体

```

1 {
2   "orders": [
3     { /* CreateOrderRequest 1 */ },
4     { /* CreateOrderRequest 2 */ }
5   ]
6 }
7

```

- `orders` 数组长度 $1 \leq N \leq 15$ ，超出返回 41015

4.3 响应体

```

1 {
2   "code": 0,
3   "msg": "ok",
4   "data": {
5     "items": [
6       {
7         "client_order_id": "cli_20260509_a",
8         "code": 0,
9         "tickets": [ { /* OrderView */ } ]
10      }
11    ]
12  }
13 }
14

```

```

10     },
11     {
12         "client_order_id": "cli_20260509_b",
13         "code": 41003,
14         "msg": "odds changed, please accept latest quote"
15     }
16 ]
17 }
18 }
19

```

- 单条失败不影响其他条
- 客户端通过 `items[i].code == 0` 判断单条成功

5. 撤单

5.1 接口

```

1 POST /api/v1/portfolio/orders/cancel
2 Content-Type: application/json
3

```

5.2 请求体 (三种模式互斥)

字段	类型	说明
<code>account_id</code>	int64	必填
<code>ticket_id</code>	int64	模式 A：按内部订单 ID 撤
<code>client_order_id</code>	string	模式 B：按客户端幂等键撤
<code>order_group_id</code>	string	模式 C：整组撤； Single 多笔下单后整体回滚常用此模式

5.3 撤单业务规则

1. 必须 `biz_state=Pending` (已结算或已撤过的不可再撤 → `41008`)
2. 必须在 `cancel_window_sec` 之内 (默认 60 秒, 配置 `bet.cancel_window_sec`)
3. 所有关联 leg 的市场必须仍 `Open`
4. 所有关联比赛必须仍 `Scheduled`
5. 调用链上提交 `BetCancel` 交易; 同步只返回 `Submitted`, 最终 `Confirmed` 由扫块回流推动 `biz_state` → `Cancelled`

5.4 示例

按 `client_order_id` 撤：

```
1 {
2   "account_id": 1001,
3   "client_order_id": "cli_20260509_001"
4 }
5
```

整组撤：

```
1 {
2   "account_id": 1001,
3   "order_group_id": "grp_cli_20260509_002"
4 }
5
```

整组撤的响应：

```
1 { "code": 0, "msg": "ok", "data": { "cancelled": 3 } }
2
```

6. 查询

6.1 单查

```
1 GET /api/v1/portfolio/orders/get?account_id=1001&ticket_id=7283901020001
2 GET /api/v1/portfolio/orders/get?account_id=1001&client_order_id=cli_202
3
```

返回单条 `OrderView`，未找到返回 `41010`。

6.2 列表查询

```
1 GET /api/v1/portfolio/orders
2
```

Query 参数	类型	说明
<code>account_id</code>	int64	必填
<code>status</code>	enum	<code>resting</code> / <code>canceled</code> / <code>executed</code>
<code>order_group_id</code>	string	同组过滤
<code>market_id</code>	int64	单市场过滤
<code>match_id</code>	int64	单比赛过滤

cursor	int64	上一页最末 order_id ，倒序游标分页
limit	int	默认 50，最大 200

返回 OrderView[] ，按 order_id 倒序排列。

示例：

```
1 GET /api/v1/portfolio/orders?account_id=1001&status=resting&limit=20
2
```

7. 错误码表

Code	含义	触发场景
0	OK	—
40001	参数非法	缺字段、order_type 取值非法、leg 为空、decimal_odds ≤ 1 等
40101	未授权	网关层注入 account_id 失败
40401	资源不存在	—
41001	市场未开放	市场状态 ≠ Open
41002	选项不存在	selection_id / ticker 找不到
41003	赔率已变化	decimal_odds ≠ 服务端最新报价（CAS 失败）
41004	报价过期	accepted_at 超过 quote_ttl_sec 或 expiration_ts 已过

41005	余额不足	链下镜像余额 < <code>notional_dollars</code>
41006	同场冲突	同注单 同 <code>market_id</code> 重复 / 串关同场互斥矩阵命中
41007	比赛已开始	比赛状态 ≠ <code>Scheduled</code>
41008	不可撤注	已结算 / 已撤 / 超过撤注 窗口 / 市场已关闭
41009	押注金额越界	< <code>min_stake</code> 或 > <code>max_stake</code>
41010	订单不存在	<code>ticket_id</code> / <code>client_order_id</code> 找不到
41011	链上提交失败	链上同步返回 <code>Failed</code>
41012	重复 <code>client_order_id</code>	(保留; 当前实现是直接 返回旧订单而不是报错)
41013	<code>time_in_force</code> 不支持	传入 非 <code>fill_or_kill</code>
41014	<code>action</code> 不支持	传入非 <code>buy</code>
41015	批量超限	单批 > 15
41016	串关腿数不足	<code>parlay</code> 时 <code>leg</code> < 2
41017	<code>ticker</code> 格式非法	不满足 <code>{match_id}.</code> <code>{market_code}.</code> <code>{option_key}</code> 三段
50001	系统内部错误	—

8. 状态机时序

参与方：客户端、下单服务、链上、链上事件回流。

8.1 下单阶段

步骤	触发方	动作	状态变化
1	客户端	POST /api/v1/portfolio/orders 提交注单	—
2	下单服务	执行 §3.4 的 12 步校验 (含赔率 CAS、余额预校验)	—
3	下单服务	落库为草稿订单	biz_state=Pending , chain_state=Pending
4	下单服务	调用链上 SubmitBetCreate	chain_state=Submitted , 写入 bet_tx_hash
5	下单服务	同步返回 200 OK 给客户端	对外 status=resting , outcome=""
6	链上	链上交易被打包确认	—
7	链上事件回流	触发 OnBetConfirmed(ticket_id, tx_hash) 回调	chain_state=Confirmed
8	下单服务	同步账户镜像： available -= stake , frozen += stake	—

客户端在第 5 步即可拿到回执，无需等待第 7 步；后续可通过查询接口或推送拿到 `chain_state=Confirmed`。

9. 与 Kalshi / Polymarket 的差异说明

#	差异点	我方做法	原因
①	计价单位	欧式赔率 + <code>notional_dollars</code> 为权威，Kalshi 的 <code>yes_price / count</code> 为派生	体育盘口业内共识；Kalshi 的 0-99 美分会损失精度（赔率 1.85 ↔ 53.x 美分被 round 成 54）
②	下单方向	仅 <code>action=buy</code>	Phase 1 没有二级市场，平台坐庄，无 SELL 场景
③	订单类型	仅 <code>time_in_force=fill_or_kill</code>	没有订单簿，挂不住，无 GTC/IOC 概念
④	串关 (Parlay)	新增 <code>order_type=parlay</code> ，赔率连乘，任一 Lost 整单输	Kalshi 的 <code>order_group_id</code> 仅"原子组"，不做连乘；体育行业必须有
⑤	<code>outcome</code> 字段	订单上直接给 <code>won/lost/refunded/cancelled/rejected</code>	Kalshi 的订单簿模型靠仓位看盈亏，我们 Phase 1 没有仓位概念
⑥	ticker 格式	三段 <code>{match_id}.{market_code}, {option_key}</code>	

		<code>{market_code}</code> . <code>{option_key}</code>	在不同比赛复用，必须带 <code>match_id</code>
⑦	批量上限	15 单 / 批	与 Kalshi/Polymarket 对齐
⑧	心跳保活	不实现	Kalshi 心跳是为了保活订单簿挂单；我们没有挂单
⑨	链上签名	服务端持有钱包，单边签名	不走 Polymarket 的 EIP-712 客户端签名
⑩	鉴权	Phase 1 网关注入 <code>account_id</code> ；Phase 2 迁移 RSA-PSS	渐进对齐

10. 配置项 (`config.yaml` → `bet`)

字段	默认	说明
<code>cancel_window_sec</code>	60	创建后允许撤单的窗口秒数
<code>min_stake</code>	1	单注最小金额
<code>max_stake</code>	100000	单注最大金额
<code>quote_ttl_sec</code>	30	<code>accepted_at</code> → 现在的最大间隔；超过即报价过期